

PYTHON 2

- Utilisation de dictionnaires -

I. Le type dict

Une variable `D` de type `dict` en Python (ou « dictionnaire » en français) est un ensemble fini de paires d'éléments donné entre accolades, les différentes paires d'éléments étant séparées par une virgule.

Chaque paire d'éléments d'un dictionnaire est définie sous la forme `u:v`, où `u` et `v` sont deux variables Python de type quelconque, ou des valeurs concrètes de type quelconque.

Pour chaque paire `u:v` du dictionnaire, `u` est appelé une « clé », et `v` est la « valeur correspondante à la clé `u` ».

Une liste en Python est une liste ordonnée d'éléments, et ces éléments sont indexés par leur rang (ou leur position) dans la liste. Un dictionnaire en Python est une liste de paires d'éléments, et ces éléments sont indexés par leur clé associée.

Les clés d'un dictionnaire ne sont pas modifiables, mais la valeur associée à une clé peut être modifiée. Les clés d'un dictionnaire doivent être deux à deux distinctes.

Exemples :

Voici quelques exemples de dictionnaires :

(1)

```
Nombre_roues={"vélo" : 2, "tricycle" : 3, "voiture" : 4}
```

Ici, le dictionnaire nommé `Nombre_roues` a trois éléments, et :

- les trois clés du dictionnaire sont les trois chaînes de caractères : `'vélo'`, `'tricycle'` et `'voiture'`
- pour chacune des clés, la valeur associée est un entier, qui donne concrètement le nombre de roues de l'élément décrit par la clé.

(2)

```
Chiffres={0 : "zéro", 1 : "un", 2 : "deux", 3 : "trois", \
4 : "quatre", 5 : "cinq", 6 : "six", 7 : "sète", \
8 : "huit", 9 : "neuf"}
```

Nous remarquons ici que l'élément du dictionnaire « 7 » est mal défini. On peut corriger comme suit, en modifiant la valeur associée à la clé 7 :

```
Chiffres[7]= "sept"
```

(3)

```
Kholle={"G1":["Léna","Kevin","Anne"],\
"G2":["Ava","Léa","Héloïse"],"G3":["Iris","Loïc","Rapahël"]}
print(Kholle["G1"])
```

Après compilation des commandes, la console affiche :

```
runcell(1, 'C:/Users/ccarc/Documents/intro_python.py')
['Léna', 'Kevin', 'Anne']
```

Sur cet exemple, nous avons mis en lumière le fait que « les éléments d'un dictionnaire sont indexés par leur clé associée », contrairement à une liste Python où les éléments sont indexés par leur rang.

On peut ajouter de nouveaux éléments à un dictionnaire. Par exemple, les instructions ci-dessous ajoute un groupe de khôlle n° 4, à ceux déjà définis :

```
Kholle["G4"]=["Victor","Maxence","Noé"]
```

(4)

```
Effectifs={}
# On a crée un nouveau dictionnaire Python, nommé Effectifs
# Puis, on définit les éléments de ce dictionnaire
Effectifs["Y2"]=108
# ici, on a défini une nouvelle clé : "Y2"
# et la valeur associée à cette clé est : 108
Effectifs[3]='107 étudiants'
print(Effectifs)
```

Après complition des commandes, la console affiche :

```
runcell(2, 'C:/Users/ccarc/Documents/intro_python.py')
{'Y2': 108, 3: '107 étudiants'}
```

Notons qu'avec cette méthode de construction d'un dictionnaire, on pourrait définir le dictionnaire `Chiffres` comme suit :

```
Chiffres={}
Ecrit=["zéro", "un", "deux", "trois", "quatre", \
      "cinq", "six", "sept", "huit", "neuf"]
for i in range(10):
    Chiffres[i]=Ecrit[i]
```

II. Quelques opérations sur les dictionnaires

On suppose prédéfinis :

- D une variable Python de type `dict`,
- c l'une des clés du dictionnaire D,
- c1 et x deux variables Python de type quelconque, c1 n'étant pas une des clés de D.

La commande Python	a pour effet de :
<code>len(D)</code>	renvoyer le nombre de clés de D
<code>u=D[c]</code>	affecter à la variable Python u la valeur associée à la clé c du dictionnaire D.
<code>D[c]=x</code>	modifier la valeur associée à la clé c, en la remplaçant par x.
<code>del D[c]</code>	supprimer du dictionnaire D la clé c et sa valeur associée
<code>D[c1]=x</code>	ajouter un nouvel élément au dictionnaire D : la paire c1:x
<code>L=D.keys()</code>	affecter à L la liste (de type <code>dict_keys</code>) des clés de D
<code>L=D.values()</code>	affecter à L la liste (de type <code>dict_values</code>) des valeurs de D
<code>L=D.items()</code>	affecter à L la liste (de type <code>dict_items</code>) des paires (clé , valeur) définissant D
<code>x in D</code>	renvoyer un booléen : True si x est bien une clé du dictionnaire, et False sinon.

Notons que si L est de type `dict_keys`, ou `dict_values`, ou `dict_items`, la commande `L=list(L)` permet de transformer L en une variable Python de type `list`.

Exercice 1 :

Reprenons l'exemple du dictionnaire donnant les chiffres écrits en toutes lettres :

```
Chiffres={0 : "zéro", 1 : "un", 2 : "deux", 3 : "trois", \
4 : "quatre", 5 : "cinq", 6 : "six", 7 : "sept", \
8 : "huit", 9 : "neuf"}
```

Ecrire un script Python qui renvoie le nombre de chiffres entre 0 et 9 qui s'écrivent en français avec exactement 4 lettres.